

משימת ניתוח מערכת: "הנאוטילוס והמכונה של נמו"

 **סיפור הרקע** קפטן נימו מהנאוטילוס הגיע למשרדך עייף ובמצב רוח קודר. הוא מספר שכאשר הוא שט בימים, מגיעים פתאום זרמי מים לא צפויים, חובטים בנאוטילוס והוא נתקע בסלעים – דבר שמבזבז זמן יקר על שיפוצים. הוא מבקש שתחזה עבורו את מפת הזרמים כדי שיוכל לנווט בבטחה.

לפתע, בעודכם משוחחים, נפתח פורטל זוהר על השולחן וממנו קופץ **קפטן נמו זעיר וצעיר** וצועק: "עצרו הכל! פתרתי את הבעיה!". נמו המבוגר המום: "מאיפה קפצת? ולמה אתה כזה פצפון?". הצעיר מנפנף בידו בביטול: "זה לא חשוב בכלל! אתה פצפון! לאן הגעת בכלל?! חיפשתי אותך על כוכב פלוטו, חשבתי שתלך לקורפלוטוניקס! בכל מקרה, פיזרתי חיישנים שמודדים זרמים, והמצאתי מכונה שמוציאה מפת זרמים בכל העולם!".

נמו המבוגר מעיף בו מבט מזלזל ואומר: "שטויות! לחזות את הזרמים בכל העולם בכל רגע נתון זו משימה קשה ביותר, אחרת הייתי כבר פותר אותה מזמן!"

נמו הצעיר שולף טאבלט קטנטן: "הנה! זו המכונה!" הוא מציג מפה ומראה שם נקודה אדומה. "פה פיזרתי 6000 חיישנים! אני אוסף את כל המידע וחוזר את הזרמים!"

נמו המבוגר מוציא ערימה של מפות נייר מהתיק שלו, משווה למפות על הטאבלט ורוטן: "דווקא פה? חשבתי שאני חכם אם המצאתי את הפיתרון לבעיה שבאתי לבקש משוכן היבשה הזה לפתור, אבל מסתבר שאני... או לפחות אני הצעיר לא כזה חכם! כנראה שגם האינטליגנציה שלך התכווצה יחד עם הגוף! מה עם כף התקווה הטובה? מה עם מיצרי גיברלטר?". הצעיר עונה בחוצפה: "המכונה שלי מבינה מהר, לא כמוך!" פתאום, הצעיר שולף שלט, מכוון למבוגר ולוחץ. הקפטן המבוגר מתכווץ לגודל של זרת! במאמץ נואש הוא מטפס על השולחן, תופס עט שמרגיש לו כמו עמוד עץ, ומתחיל לחבט בצעיר. השניים מתרוצצים לך על השולחן, דוחפים את כוס הקפה שלך, חובטים זה בזה, נמו המבוגר זורק חפצים על נמו הצעיר והצעיר בורח תוך שהוא מנופף בטאבלט שלו. הם מייצרים מהומה על שולחן העבודה שלך, ורצים על המקלדת תוך שהם צורחים אחד על השני בהינדי.

המטרה שלך היא להבין האם המכונה שהמציא נמו הצעיר תציל את הנאוטילוס או תוביל אותה לאבדון!

שלב 1 - איסוף מידע. לפני שאתה נוגע במכונה, עליך להבין אם הנתונים שנמו הצעיר אסף בכלל הגיוניים. אתה מוזמן לשאול את שני הקפטנים (המראיין) כל שאלה על מהות הבעיה הפיזיקלית ועל הנתונים ששימשו לבניית המודל.

🚩 שלב 2: ארכיטקטורה ו-Loss

👤 **סיכום שלב 1 (מוקרא למועמד):** "אחרי שחצצת בין הקפטנים (והצלת את העט שלך מידיו של נמו המבוגר שהשתמש בו כחנית), התמונה מתבהרת: נמו הצעיר בנה מודל שמתבסס על נתונים חלקיים מאוד. הוא אסף מידע רק במים רדודים, בקיץ, והתעלם מהעומק, מהמליחות ומהטופוגרפיה של קרקעית הים. המכונה שלו חכמה, אבל היא ראתה רק חלק קטן מאוד מהאוקיינוס. כמו כן למדת כי יש חוסר במידע אמיתי לאימון המערכת. לאחר מאמץ משותף סיכמתם על התיקונים שיש באיסוף המידע לצורך אימון מדויק יותר של המערכת."

הנחיה למועמד: "נמו הצעיר מראה לך את השרטוט של המכונה. זו רשת MLP פשוטה (Dense Layers) שמקבלת מיקום (x, y) ופולטת עוצמת זרם. הוא משתמש ב-MAE כפונקציית הפסד. איך תשנה את המבנה והמדדים כדי שהמכונה תבין את המרחב והזמן של הים?"

שלב 3:

🤖 **סיכום שלב 2 (מוקרא למועמד):** "נמו הצעיר נעלב מהשינויים שלך בארכיטקטורה וצועק: 'למה צריך CNN? המכונה שלי זוכרת הכל בעל פה!'. נמו המבוגר, בינתיים, בנה לעצמו סירה מנייר ושט בתוך שלולית הקפה שנשפכה, צועק לך: 'הילד הזה לא מבין בבטיחות! ה-Loss שלו יהרוג אותנו!'. הבנת שהמכונה צריכה 'עיניים' מרחביות וזיכרון לטווח ארוך כדי לא להתרסק."

🚩 שלב 3: קוד

הנחיה למועמד: "לפניך הקוד של נמו הצעיר. אנא חווה דעתך. ציין דגשים שיש לבצע בעת בחינת קוד ושינויו. לאחר מכן בצע את השינויים. אתה רשאי להשתמש בכל כלי שהוא אך הקפד לבחון את התוצאה ולהסביר את השיקולים שלך לגבי השינויים שתבצע."

קח בחשבון שנמו המבוגר מתלונן על כך שיש הבדל גדול בדיוק המודל במיקומים סמוכים זה לזה, גם לאחר שאומנה עם מידע חדש בשלב שניתחת בשלב 1."

```
def get_data_for_coords(sensors_list, target_lat, target_lon):  
  
    """  
    פונקציה הסורקת את כל רשימת החיישנים  
    כדי למצוא נתונים עבור קואורדינטה ספציפית  
    """  
  
    # נמו הצעיר שומר את החיישנים כרשימה פשוטה של אובייקטים  
  
    data = {}  
  
    for sensor in sensors_list:  
  
        if sensor.lat == target_lat and sensor.lon == target_lon:  
  
            # מחזיר וקטור של נתונים: [טמפרטורה, צפיפות, מהירות]  
  
            data[sensor.type] = [sensor.value])  
  
    # אם לא נמצא אף חיישן בדיוק בנקודה, מחזירים רשימה ריקה  
  
    return data
```

```

def analyze_ocean_safety(world_grid, sensors):
    prev_prediction = 0
    dangerous_areas = []

    for lat in range(len(world_grid.lats)):
        for lon in range(len(world_grid.lons)):
            model = load_nemo_model("nemo_v1.h5")

            # שליפת נתונים לכל קואורדינטה
            current_data = get_data_for_coords(sensors, lat, lon)

            # חיזוי לנקודה בודדת
            prediction = model.predict(current_data)

            # לוגיקת קשר בין נתונים: בדיקת הפרש מהשכך הקודם
            if abs(prediction - prev_prediction) > 0.5:
                dangerous_areas.append((lat, lon, prediction))

            prev_prediction = prediction

    return sorted(dangerous_areas, key=lambda x: x[2], reverse=True)

```

שלב 3 ב

הקפה שנשפך הרס חצי מהזיכרון של המחשב. עליך לשכתב את הלוגיקה כך שתעבוד מהר ועם מעט RAM

🤖 **סיכום שלב 3 (מוקרא למועמד):** "בזמן שהקפטנים מנסים להשתלט על העכבר שלך, הצלחת לייעל את הקוד. עכשיו המכונה לא 'נחנקת' מהמידע אלא מעבדת אותו בזרם חכם. נמו המבוגר מתרשם מהחיסכון בזיכרון וממלמל: 'אולי יש עתיד למכונות האלו, אם הן נכתבות על ידי מישהו עם שכל'."

בזכות ה-Generator, הנאוטילוס מצליחה לעבד את מפת הזרמים בזמן אמת. אתה רואה על המסך איך הנתונים זורמים ב-Batches מסודרים, והזיכרון של המחשב נשאר יציב למרות נזקי הקפה. עכשיו כשיש לנו מהירות, בוא נראה אם המפות שלך באמת שוות משהו, או שאנחנו בדרך להתנגשות מהירה במיוחד."

🚩 **שלב 4: בחינת הפתרון בזמן אמת**

הנחיה למועמד: "נמו הצעיר חוגג: 'יש לי 99% דיוק במפות!'. איך תבנה תהליך בדיקה שיבטיח שהנאוטילוס לא תתרסק כשתגיע לאזור חדש לגמרי?"

שאלת סיכום:

בסוף המשימה, אחרי שהוא פותר את הכל, אתה יכול לשאול בחיור: "קפטן נמו המבוגר מסתכל על הקוד שלך ואומר: 'זה יפה, אבל מה יקרה אם מחר נגלה זן חדש של לווייתנים שמשפיע על הזרמים? נצטרך לאמן את כל המכונה מחדש?'"

דף למראיין:

רקע: הקרא או תן למועמד את הדף עם סיפור הרקע

● **שלב 1: שלב החקירה (השאלות לקפטן נמו)**

הסיטואציה: המועמד מנסה להבין את הבעיה. אתה משחק את שני הקפטנים נמו המבוגר – הוא חכם, מנוסה, אבל מתוסכל מהטכנולוגיה ה"פשוטה" של הצעיר. הצעיר - נמהר, שחצן, מכיר את מבנה המכונה והנתונים שנאספו.

שאלות הכוונה (אם המועמד נתקע):

1. "האם היית סומך על מודל שחזזה מזג אוויר בחורף אם הוא אומן רק בקיץ?"
2. "האם זרם המים בחוף הים הזה לזרם בעומק של 5 קילומטרים?"
3. "נמו המבוגר אומר שיש אזורים עם סלעים ויש אזורים פתוחים. איך זה משפיע על המידע שנצטרך?"
- מקרים ותגובות:

שאל על... אם המועמד	התשובה של נמו המבוגר	המשמעות המקצועית (מה שאתה מחפש)
מיקום גיאוגרפי	צעיר - "פיזרתי 6000 חיישנים ליד המעבדה שלנו" מבוגר - "הנאוטילוס מפליגה בכל העולם, אבל הצעיר הזה שם חיישנים רק ליד המעבדה שלו באי בהאמה!" צעיר - "המים שם נעימים! נסה אתה לצאת לשים חיישנים בקוטב!"	Out of Domain / Data Drift. (מה שאתה מחפש)
עומק הצלילה	מבוגר - "אני יורד לעומקים של 10,000 מטר ואפילו עד ל 20000 מיל מתחת לפני הים!!! הזרמים שם שונים לגמרי מהזרמים על פני השטח."	Feature Engineering (Z-axis).

	צעיר - "לא אכפת לי מעומקים! אני מראה את הזרמים הממוצעים בנקודות גיאגרפיות על הגלובוס!"	
Temporal Bias / Seasonality	צעיר - "ביציתי את המדידות בקיץ, כאשר בן הדוד שלי היה בחופשה מבית הספר ויכול היה לעזור לי, אבל החיישנים שולחים מידע בכל רגע נתון, גם בחורף!" מבוגר - "המכונה למדה בקיץ וחוזה את הזרמים בחורף? באיזה עולם שטת?!! המח שלך הפליג יחד עם הנאוטילוס והשאיר אותך מאחור??"	זמן ועונתיות
.Missing Features	מבוגר - "הוא מודד רק מהירות מים. הוא שכח שמליחות (Salinity) וטמפרטורה משנות את צפיפות המים וגורמות לשינויים מהותיים בזרמים!" צעיר - "עזוב שטויות. איתך היה מדען בצוללת ששיגע אותך קצת."	סוגי נתונים/פיזיקה
	מבוגר - "איך בכלל למדת את המכונה?" צעיר - "כל פעם שהמכונה חזתה זרם זרקתי את אחד המלחים למים לוודא שהוא נסחף בכיוון שהיא חזתה" מבוגר - "מה?!!? נשאר לך בכלל מלחים?" צעיר - "מי בכלל צריך מלחים בעולם של בינה מלאכותית?!!?"	GT

🔧 שלב 2: פירוק ה"מכונה" (ניתוח הרשת)

הסיטואציה: המועמד בוחן את ה-Black Box של נמו הצעיר.

התקלה המובנית: נמו הצעיר בנה רשת MLP (דנסית) פשוטה. היא מקבלת קואורדינטה (x, y) ופולטת וקטור זרם.

- **הבעיה:** הרשת לא "רואה" את השכנים שלה (אין CNN או GNN לקשרים מרחביים בין חיישנים) ולא זוכרת מה קרה לפני דקה (אין RNN/Transformer).
- **הקשר זמני:** הצעה להשתמש ב-LSTM / GRU או Transformer (כדי להבין זרם כרצף ולא כנקודה).

שאלות מנחות שלך:

1. "האם המכונה מבינה שמה שקורה מטר ליד הנאוטילוס משפיע עליה, או שהיא רואה כל נקודה בים כאי בודד?" (בדיקת הבנה ב-Spatial Correlation).
2. "האם יש דרך לגרום למכונה להתייחס באופן שונה למדידת הזרמים באזורים בהם יש סכנת התקלות בסלעים לעומת אזורים אחרים?" (Weighted loss function based on topography)
3. "האם המכונה יודעת שהזרם הוא תהליך מתמשך? או שהיא מנחשת כל רגע מחדש?" (בדיקת הבנה ב-Time Series). (מצופה: LSTM/Transformer).

שלב 3: אופטימיזציה ומבחן הזיכרון

הסיטואציה: הפיתרון של נמו הצעיר לוקה בשתי נקודות:

1. פרדיקציה לכל קואורדינטה בנפרד.
2. איסוף מידע לא יעיל מהחיישנים.
3. שימוש במילון לגבי כל קואורדינטה וחישון.

שאלות הכוונה:

1. "שים לב לשורה של `load_model`. כמה פעמים אנחנו טוענים אותו? האם אפשר אחרת?"
2. "הפונקציה `get_data_for_coords` רצה על כל הרשימה בכל פעם. אם יש מיליון חיישנים, מה יקרה לביצועים?" (מצופה: שימוש באינדקס/מילון).
3. "המחשב גוסס בגלל הקפה. האם אנחנו חייבים להחזיק את כל התוצאות בזיכרון בבת אחת?" (מצופה: `yield/Generator`).
4. "למה נמו המבוגר רואה קפיצות בדיוק במיקומים סמוכים? אולי חסר 'יישור קו' בין החיישנים?" (מצופה: נרמול או `Spatial Indexing`).

דף עזר למראיין (הקוד והפתרון):

מה אתה מצפה לראות בשינוי (ה-AI יכול לעזור לו לכתוב, אבל הוא חייב להסביר):

1. **Hoisting**: להוציא את `load_model` מחוץ ללולאות.
2. **Vectorization**: במקום 2 לולאות `for`, להעביר מטריצה כ-Batch אחד לרשת.
3. שינוי לוגיקת ה-Difference (האתגר האמיתי):
 - a. בלולאת ה-`for`, ההפרש נבדק מול `prev_prediction`.
 - b. ב-Batch, המועמד צריך להבין שהוא משווה וקטור לעצמו בהזזה אחת (Vectorized Shift). ב-Numpy זה נראה כך: `diff = np.abs(current_batch[1:] - [current_batch[:-1]`
4. אם נשאר עם מילון אז לבנות אותו חכם שקואורדינטה תחזיר רשימת חיישנים בנקודה. (עדיף מבני נתונים יעילים יותר). (החלפת מבני הנתונים הגלובליים מעידה על הבנה מערכתית טובה יותר)
5. חיפוש קואורדינטות על פי קרבה, למשל בעץ KD-Tree - מעיד על הבנה כי אי אפשר למפות את העולם ברציפות, ניסיון רב והכרה של שיטות מתקדמות. (Senior).
6. טיפ למראיין: אם המועמד אומר "אני אשתמש ב-Pandas או Xarray כדי לעשות את זה בשורה אחת", זה אפילו יותר טוב! זה מראה שהוא מכיר ספריות ייעודיות לנתונים גיאוגרפיים.
7. **Parallelism**: שימוש ב-GPU או Multi-threading לעיבוד הנתונים הגולמיים.
8. נרמול נתונים מהחיישנים השונים.
9. שימוש ב-dataloader או generator עם `yield` לניצול זיכרון טוב יותר.
10. קוואנטיזציה של הנתונים.
11. pruning

הקוד המצופה (בקווים כלליים):

```

# שלב 1: בונים אינדקס של אובייקטי החיישנים (פעם אחת בהרצה)

# המפתח: קואורדינטה. הערך: רשימת אובייקטים של חיישנים שנמצאים שם.

sensor_index = {}

for s in sensors_list:

    coords = (s.lat, s.lon)

    if coords not in sensor_index:

        sensor_index[coords] = []

    sensor_index[coords].append(s)

def get_batch_generator(world_grid, sensor_index, batch_size=1024):

    """
    של נתונים עדכניים בזמן אחת Batches מייצר.
    משטח את הלולאה הכפולה לרשימת נקודות אחת.
    """

    # כדי להימנע מלולאות (lat, lon) שלב 2: שיטוח רשת העולם לרשימה של
    מקוננות

    all_points = [(lat, lon) for lat in world_grid.lats for lon in
world_grid.lons]

    current_batch = []

    for coords in all_points:

        # שליפה ישירה של האובייקטים (שערכיהם מתעדכנים בנפרד)

        sensors_at_point = sensor_index.get(coords, [])

        # לוגיקה מהירה לאיסוף הנתונים העדכניים מהאובייקט

```

```

point_data = {'temp': 0, 'density': 0, 'salinity': 0}

for s in sensors_at_point:

    point_data[s.type] = s.value # הערך נלקח ישירות מהאובייקט
    # החישובים

    current_batch.append([point_data['temp'],
point_data['density'], point_data['salinity']])

    if len(current_batch) == batch_size:

        yield np.array(current_batch)

        current_batch = []

if current_batch:

    yield np.array(current_batch)

```

```

def optimized_safety_analysis(sensors_stream):

    model = load_nemo_model("nemo_v1.h5")

    # RAM-כדי לחסוך ב-Generator שימוש ב
    for batch in get_batch_generator(sensors_stream, size=1024):

        predictions = model.predict(batch)

        # לוגיקה וקטורית להחלפת הלולאה

        # משווים כל איבר בבאץ' לאיבר שלפניו בפעולה אחת

        diffs = np.abs(np.diff(predictions, prepend=prev_val))

```

```
# סינון מהיר של אזורים מסוכנים

mask = diffs > 0.5

# ... שמירת תוצאות ...
```

שלב 3ב

הסיטואציה:

יש מעט זיכרון וניסיון לטעון את כל המידע כפי שנעשה קודם גורם לקריסת המערכת כיוון שיש מעט זיכרון צריך לדעת לעבוד ב batches. שימוש ב generator יחד עם yield יכול לעשות זאת ושימוש ב dataloader עדיף

מדד ציון לשלב הקוד:

- **גרוע:** לא מבין למה `load_model` בתוך לולאה זה רע.
- **בינוני:** מוציא את המודל החוצה ומבצע Batching, אבל מסתבך בלוגיקה של ההפרשים (מנסה להריץ לולאה בתוך ה-Batch).
- **מצוין:** מזהה ש-`get_data_for_coords` היא צוואר בקבוק, משתמש ב-**Generator** לטעינת נתונים, וממיר את לוגיקת ההפרשים לחישוב וקטורי/מטריציוני.

שלב 4: ה-Loss וה-Overfit

הסיטואציה: המועמד צריך להסביר למה המכונה "טועה" על אף שמציגה דיוק גבוה.

שאלות הכוונה:

1. "אם נחלק את המידע ל-Train ו-Test בצורה אקראית, האם באמת בדקנו אזור חדש או סתם נקודות דומות?" (מצופה: Spatial Split).
2. "איך נמנע מהרשת פשוט לשנן את המיקום של 6,000 החיישנים?" (מצופה: Regularization/Dropout).
3. האם השגיאות בכל הקואורדינטות שקולות בחשיבותן?
4. איך תוודא את יציבות המודל? (בחירת נתוני קיצון בקבוצת הטסט).

מקרים ותגובות כאשר המועמד מנסה לברר את תוצאות המכונה:

התשובות של נמו הצעיר (החוצפן):

- "בבדיקה שלי (Test set) המכונה צדקה ב-99%! (אבל ה-Test שלו הוא בעצם דגימות מאותו אזור של ה-Train)." •
- "השתמשי ב-MSE (Mean Squared Error) כי זה מה שכולם עושים!" •

הכשל הלוגי שאתה רוצה שהמועמד יעלה:

- ה-Loss: אם המודל טועה ב-2 קמ"ש בלב אוקיינוס שקט, זה לא נורא. אם הוא טועה ב-2 קמ"ש ליד שונית אלמוגים, הצוללת מתרסקת. המועמד צריך להציע **Weighted Loss** (עונש כבד יותר ליד מכשולים).

שאלת סיכום:

זו שאלה על **Fine-tuning** (עדכון מודל קיים במקום אימון מאפס) – וזה סגירה מעולה לריאיון שבודקת ראייה מערכתית.



סיכום: איך להתמודד עם מועמד שנסמך רק על AI?

אם המועמד מבקש מ-ChatGPT "תכתוב לי רשת לחיזוי זרמים", הוא יקבל קוד גנרי. התפקיד שלך הוא לוודא הבנה ולשאול:

- "אוקיי", ה-AI נתן לך LSTM. אבל איך תתמודד עם זה שהחיישנים של נמו זזים עם הזרם? המיקום שלהם לא קבוע (Lagrangian sensors)."
- "ה-AI הציע CNN. איך תכניס ל-CNN את העובדה שהאדמה היא כדורית והקצוות של המפה מחוברים?"

סיכום למראיין - "מדד האיכות" של המועמד:

התנהלות המועמד	ציון
שואל על איכות הנתונים (מליחות, עומק), מציע ארכיטקטורה מורכבת (Transformer/GNN), ומזהה מיד שהקוד בזבזני בגלל חוסר בוקטוריזציה.	מצוין (Senior)
מזהה Overfit ובעיות ב-Loss, מציע CNN או LSTM, מצליח לשפר את הלולאות לקוד יעיל יותר.	טוב (Mid)
רץ ישר לפתרונות טכניים (יותר שכבות!) בלי להבין שהנתונים (Domain) חסרים, לא מצליח להסביר למה MAE אולי לא מתאים פה.	טעון שיפור

🔦 "נורות אדומות" (מתי המועמד חלש?)

- הוא רץ ישר לכתוב קוד בלי לשאול את נמו על הנתונים.
- הוא מציע רק "להוסיף עוד שכבות" כדי לפתור בעיות דיוק. או משתמש במונחים מקצועיים בלי להסביר את ההיגיון בשימוש בהם.
- הוא לא מזהה שטעינת מודל בתוך לולאה היא טעות קריטית בביצועים.
- הוא נשען על ה-AI שיכתוב לו קוד אבל לא יודע להסביר מה זה `batch_size` או `yield`.

🌟 "סימני הצטיינות" (Senior Level)

- מדבר על **Physics-Informed Neural Networks (PINNs)** – שילוב חוקי הפיזיקה ב-Loss.
- מציע **Quantization** כדי שהמודל ירוץ מהר על חומרה חלשה.
- מזהה מיד שחסרה מליחות (Salinity) כפרמטר קריטי לזרמי אוקיינוס.